FRAGMENTACIÓN POR REGIONES Y CARGA MASIVA

¿QUÉ HACE ESTA SESIÓN?

Trabaja sobre la base juego_online (Sesiones 1–4) y resuelve tres problemas con tres scripts SQL, más un paso conceptual de nube. El orden importa: primero organizas los datos (particionar), luego optimizas el acceso (indexar), después los alimentas con volumen (ETL).

Script 1 — 01_fragmentacion_particiones_region.sql

Objetivo: dividir físicamente la tabla de partidas por región (América / Europa), como si cada región viviera en su propio servidor.

QUÉ HACE EL CÓDIGO:

sql

```
CREATE TABLE Partidas_Fragmentada (
    ...
    region VARCHAR(20) NOT NULL,
    ...
    PRIMARY KEY (partida_id, region)
) ENGINE=InnoDB
PARTITION BY LIST COLUMNS(region) (
    PARTITION p_america VALUES IN ('AMERICA'),
    PARTITION p_europa  VALUES IN ('EUROPA')
);
```

PARTITION BY LIST COLUMNS(region) → MySQL guarda las filas de 'AMERICA' en p_america y las de 'EUROPA' en p_europa, aunque para el usuario siga siendo una sola tabla.

PRIMARY KEY (partida_id, region) → regla obligatoria del motor: toda clave única debe incluir la columna de partición.

Sin FOREIGN KEY → MySQL/MariaDB no permiten FKs en tablas particionadas. La integridad referencial pasa a la aplicación. Es la lección central de la sesión.

MIGRACIÓN DE DATOS:

sql

INSERT INTO Partidas_Fragmentada (...)

SELECT p.jugador_id, p.juego_id, j.region, ...

FROM Partidas p

JOIN Jugadores j ON j.jugador_id = p.jugador_id;

La región no está en Partidas; se toma del jugador (j.region). Esto denormaliza el dato a cambio de poder particionar.

VERIFICACIÓN:

sql

EXPLAIN SELECT * FROM Partidas_Fragmentada WHERE region = 'AMERICA';

La columna partitions del EXPLAIN debe mostrar solo p_america → partition pruning: MySQL nunca abre la partición que no necesita.

Script 2 — 02_indices_matchmaking.sql

Objetivo: que la consulta de matchmaking (buscar jugadores de nivel similar en la misma región) responda en milisegundos en vez de escanear miles de filas.

LOS TRES ÍNDICES:

sql

CREATE INDEX idx_partidas_frag_region_fecha

    ON Partidas_Fragmentada (region, fecha DESC);   -- filtro por región + orden por fecha

```
CREATE INDEX idx_partidas_frag_jugador
    ON Partidas_Fragmentada (jugador_id);          -- acelera el JOIN

CREATE INDEX idx_jugadores_region_nivel
    ON Jugadores (region, nivel);                  -- filtro por región + rango de nivel
```

Regla de diseño: en un índice compuesto, primero igualdad (region), luego rango u ordenación (nivel / fecha).

La consulta de matchmaking:

sql

```
EXPLAIN
SELECT j.jugador_id, j.nombre_usuario, j.nivel, MAX(p.puntaje) AS mejor_puntaje_reciente
FROM Jugadores j
JOIN Partidas_Fragmentada p ON p.jugador_id = j.jugador_id AND p.region = j.region
WHERE j.region = 'AMERICA'
  AND j.nivel BETWEEN 10 AND 15
GROUP BY j.jugador_id, j.nombre_usuario, j.nivel
ORDER BY mejor_puntaje_reciente DESC
LIMIT 10;
```

AND p.region = j.region en el JOIN es clave: propaga el partition pruning también al JOIN (solo abre p_america).

BETWEEN 10 AND 15 es el corazón del matchmaking por habilidad.

EXPLAIN debe mostrar key poblada y type distinto de ALL → el índice se está usando.

Nota: CREATE INDEX IF NOT EXISTS no existe en MySQL (solo MariaDB). Si se re-ejecuta el script, hay que hacer DROP INDEX antes para evitar el error Duplicate key name.

Script 3 — 03_etl_carga_masiva.sql

Objetivo: simular la importación de un CSV con miles de partidas históricas, usando una CTE recursiva (equivalente educativo a LOAD DATA INFILE).

GENERACIÓN DE 5000 FILAS:

sql

```
INSERT INTO Partidas_Fragmentada (...)
WITH RECURSIVE secuencia AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM secuencia WHERE n < 5000
)
SELECT
    j.jugador_id,
    FLOOR(RAND() * 2) + 1 AS juego_id,        -- juego 1 o 2
    j.region,                                 -- región del jugador
    FLOOR(RAND() * 9000) AS puntaje,          -- 0 a 8999
    FLOOR(RAND() * 900) + 60 AS duracion_seg,  -- 60 a 959 seg
    ELT(FLOOR(RAND() * 2) + 1, 'victoria', 'derrota'),
    NOW() - INTERVAL FLOOR(RAND() * 180) DAY
FROM secuencia
JOIN Jugadores j
    ON j.jugador_id = ((secuencia.n MOD (SELECT COUNT(*) FROM Jugadores)) + 1);
```

DESGLOSE:

CTE recursiva secuencia → genera los números 1 a 5000 (caso base 1, caso recursivo n + 1).

RAND() → solo afecta a puntaje, duración, resultado y fecha (no a jugador ni región).

MOD COUNT(*) → reparte las 5000 filas en ciclo entre los 5 jugadores (1000 cada uno). Como 3 son de América y 2 de Europa → 3000 / 2000 exactos, siempre.

VERIFICACIÓN:

sql

SELECT region, COUNT(*) AS total_partidas

FROM Partidas_Fragmentada

GROUP BY region;

Resultado esperado: AMERICA = 3000, EUROPA = 2000.

Nota MySQL 8: si aparece error de recursión, descomentar SET SESSION cte_max_recursion_depth = 10000; (MySQL limita a 1000 niveles por defecto; MariaDB no).

Paso 4 — Comandos de terminal (no SQL)

Solo referencia conceptual. No se pegan en phpMyAdmin.

bash

mysqldump -u root -p juego_online Partidas_Fragmentada > historico_partidas.sql

aws s3 cp historico_partidas.sql s3://mi-bucket-juego/backups/

aws s3 cp s3://mi-bucket-juego/import/nuevas_partidas.csv ./nuevas_partidas.csv

mysql -u root -p juego_online -e "LOAD DATA INFILE './nuevas_partidas.csv' INTO TABLE Partidas_Fragmentada ..."

Carga (upload): exportar con mysqldump y subir a S3/Azure.

Descarga (download): bajar un CSV de la nube y cargarlo con LOAD DATA INFILE o la pestaña Importar de phpMyAdmin.

Es el mismo patrón Extract → Transform → Load del Script 3, solo que el origen/destino es almacenamiento remoto.

Orden de ejecución (obligatorio)

text

Script 1 → crea Partidas_Fragmentada y migra

Script 2 → indexa la tabla creada

Script 3 → inserta 5000 filas en la tabla indexada

Si se altera el orden, los scripts fallan (tabla inexistente) o se pierden datos (el DROP TABLE del Script 1 borra las 5000 filas si se vuelve a ejecutar).

EN RESUMEN**:**

PARTITION BY LIST COLUMNS(region) divide físicamente por región; el usuario ve una sola tabla.

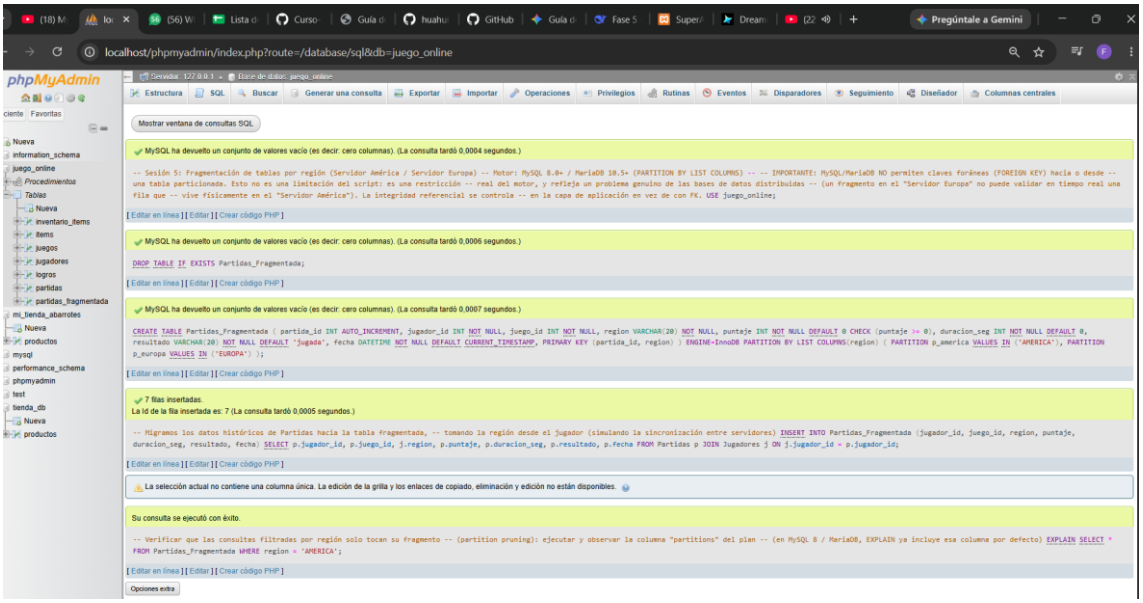
La PK debe incluir la columna de partición (partida_id, region) — regla del motor.

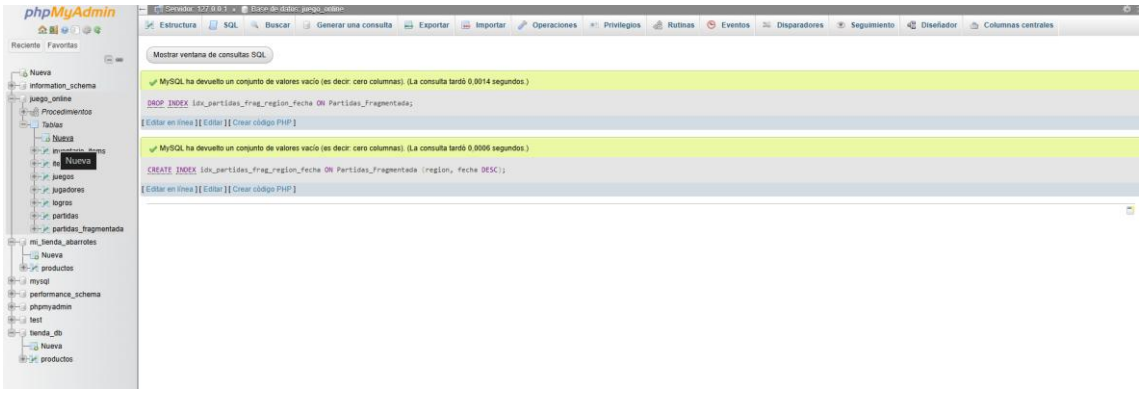
Sin FKs en tablas particionadas → la integridad se controla en la aplicación.

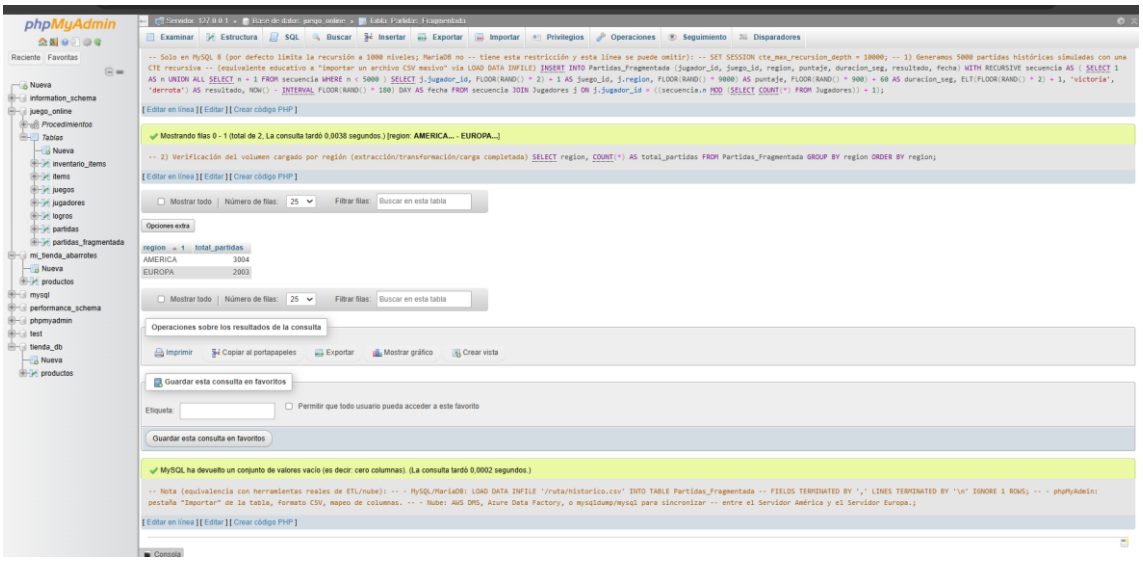
Índices compuestos con orden igualdad → rango/ordenación transforman type: ALL en type: ref/range.

CTE recursiva genera 5000 filas sin archivos externos; MOD garantiza el reparto 3000/2000.

El patrón ETL es el mismo en local (phpMyAdmin) y en la nube (S3, Azure, Glue).


phpMyAdmin
localhost/phpmyadmin/index.php?route=/database/sql&db=juego_online
Mostrar ventana de consultas SQL
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0004 segundos.)
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0006 segundos.)
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0007 segundos.)
7 filas insertadas.
La id de la fila insertada es: 7 (La consulta tardó 0,0005 segundos.)
La selección actual no contiene una columna única. La edición de la grilla y los enlaces de copiado, eliminación y edición no están disponibles.
Su consulta se ejecutó con éxito.


phpMyAdmin
Mostrar ventana de consultas SQL
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0014 segundos.)
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0006 segundos.)


phpMyAdmin
Mostrando filas 0 - 1 (total de 2, La consulta tardó 0,0038 segundos.) [region: AMERICA... - EUROPA...]
region total_partidas
AMERICA 3004
EUROPA 2003
Operaciones sobre los resultados de la consulta
Guardar esta consulta en favoritos
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0002 segundos.)